blur.consulting

blur.consulting

Blur Consulting

@BlurConsultingLatam

LIVE

YA COMENZAMOS

BLUR CONSULTING

GRUPO DE ESTUDIO · DATA ANALYST

Sesión 3: Uniendo tablas: JOINs, UNION y modelo estrella



BLUR CONSULTING

Proyecto Final
Modulo SQL

Objetivo del proyecto

Desarrollar un caso de negocio que permita aplicar los conocimientos de SQL adquiridos durante las sesiones, mediante la limpieza, validación, transformación y análisis de datos, con la finalidad de construir un modelo dimensional tipo estrella que facilite el análisis de información y la toma de decisiones empresariales.

Organización de los equipos

- ❑ Los equipos estarán conformados por 3 integrantes.
- ❑ Cada equipo deberá seleccionar una fuente de datos de Kaggle u otra fuente confiable.
- ❑ A partir de los datos seleccionados, deberán identificar una problemática o necesidad de negocio que pueda resolverse mediante el análisis de datos.
- ❑ Todos los integrantes deberán participar en el desarrollo del proyecto y en la exposición final.



BLUR CONSULTING

Proyecto Final
Modulo SQL



Desarrollo del proyecto

Cada equipo deberá desarrollar un caso de negocio que contemple los siguientes aspectos:

- ❑ Planteamiento del caso de negocio: identificar una problemática empresarial, describir el contexto y definir qué se busca analizar o resolver.
- ❑ Selección de la fuente de datos: obtener un conjunto de datos pertinente para el caso elegido, documentando su procedencia.
- ❑ Limpieza y validación de datos: identificar y tratar valores nulos, registros duplicados, inconsistencias y problemas en los tipos de datos, según corresponda.
- ❑ Diseño del modelo dimensional: construir un modelo tipo estrella, definiendo la tabla de hechos, sus dimensiones, las claves y las relaciones entre ellas.
- ❑ Transformación y análisis con SQL: desarrollar consultas que permitan preparar los datos, relacionar tablas y generar información útil para el negocio.
- ❑ Cálculos e indicadores: implementar las métricas necesarias para responder las preguntas del negocio, tales como montos, cantidades, promedios, porcentajes u otros indicadores relevantes.
- ❑ Justificación de la solución: explicar por qué se eligió ese modelo, cómo se desarrolló y de qué manera contribuye a resolver la problemática identificada.

BLUR CONSULTING

Proyecto Final
Modulo SQL



Entregables finales

La presentación en PowerPoint (PPT) y el archivo SQL deberán cargarse en el apartado “Proyecto Final del Módulo”, ubicado en la sesión 4 del Classroom. La entrega estará habilitada hasta el 1 de noviembre de 2026 a las 11:59 p. m. (hora de Perú).

La presentación deberá incluir:

- ❑ Portada con el título del proyecto y los integrantes del equipo.
- ❑ Descripción del contexto del negocio (ficticio).
- ❑ Planteamiento del problema y objetivo del análisis.
- ❑ Fuente de datos utilizada y descripción de los datos.
- ❑ Proceso de limpieza, validación y transformación.
- ❑ Diseño del modelo estrella, incluyendo la tabla de hechos y sus dimensiones.
- ❑ Explicación de las consultas SQL y los cálculos realizados.
- ❑ Presentación de los principales resultados obtenidos.
- ❑ Justificación de la solución propuesta y su aporte al negocio.

¿Quíenes somos?



BLUR
CONSULTING

Somos una consultora especializada en servicios tecnológicos, desarrollo de proyectos, provisión de talento y formación profesional en Ingeniería, Gobierno y Analítica de Datos.

Misión:

Impulsar la transformación digital y el rendimiento estratégico de las organizaciones mediante soluciones integrales en ingeniería y analítica de datos, respaldadas por la capacitación constante y la provisión del mejor talento técnico del mercado..

Visión:

Ser la consultora y aliada estratégica líder en gobierno, arquitectura y analítica de datos en la región, reconocida por la excelencia técnica de nuestras soluciones, la calidad de nuestros programas de formación y nuestra capacidad para potenciar equipos de alto rendimiento.

Redes Sociales

blur.consulting

@BlurConsultingLatam

blur.consulting

Blur Consulting



CONOCE AL PONENTE :

Jesús Evangelista Azañero



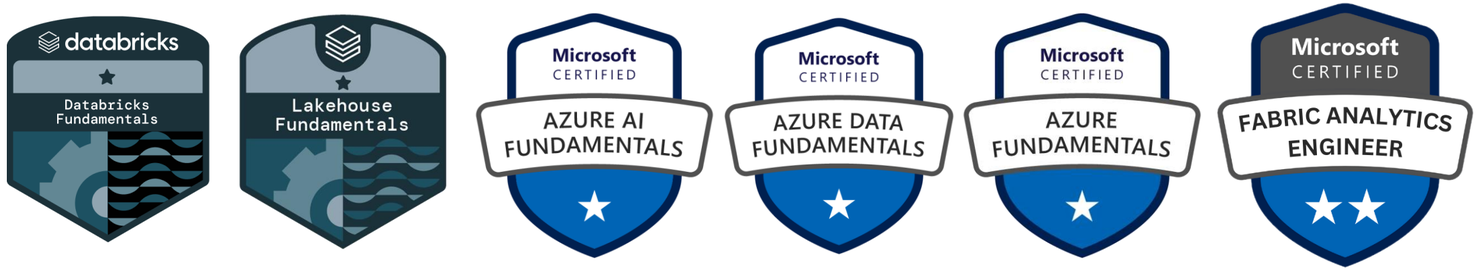
Formación :

- Ingeniero de Sistemas - UNJFSC
- Experiencia en Análisis de Datos y procesos ETL.
- Conocimiento en Python, SQL, PySpark, Azure, Databricks.
- Formación universitaria orientada al desarrollo de soluciones tecnológicas y gestión de datos.

Exp. Laboral :

kyndryl YANBAL BCP

Certificaciones :



Agenda

01 **INNER JOIN: unir tablas**

02 **Tipos de JOIN: encontrar lo que falta**

03 **JOIN de varias tablas**

04 **UNION y UNION ALL**

05 **Modelamiento dimensional**

06 **Caso integrador y práctica**

“Un dato suelto informa; unido a los demás, explica.”



BLUR CONSULTING

¿Qué lograrás hoy?

- **Unir tablas con INNER JOIN**
  Traer a cada pedido el nombre del territorio, del producto o del cliente.
- **Encontrar lo que falta con LEFT I RIGHT JOIN**
  Clientes que nunca compraron y productos que nunca se vendieron.
- **Encadenar varias tablas**
  Ir del detalle del pedido hasta la categoría, sin inflar las ventas.
- **Apilar resultados con UNION o UNION ALL**
  Juntar canales, armar reportes por partes y comparar listas de clientes.
- **Modelar como en un Data Warehouse**
  Kimball vs Inmon, y modelo estrella vs copo de nieve: la base de Power BI.



BLUR CONSULTING

La información está repartida

Sales.SalesOrderHeader (pedidos)

SalesOrderID	TerritoryID	TotalDue
43659	5	23,153.23
43660	5	1,457.33
43661	6	36,865.80



Sales.SalesTerritory (zonas)

TerritoryID	Name	País
4	Southwest	US
5	Southeast	US
6	Canada	CA

¿Cuánto vendió Southeast?

Podemos indicar que tiene 2 registros en pedidos y cuyo valor sumaría a 24,610.56

La columna en común (TerritoryID) es la llave que conecta las dos tablas.

1 JOIN

Une tablas de costado usando una columna en común (la llave).

2 UNION

Pega resultados uno debajo del otro. Agrega filas: las ventas en tienda y las de delivery forman una sola lista. Ambas consultas deben tener las mismas columnas y en el mismo orden.

3 Modelo

Organiza las tablas para que el análisis sea fácil. En el centro va una tabla de hechos con lo que se mide (ventas, montos) y alrededor, las dimensiones con el contexto (quién, qué, dónde, cuándo).

BLUR CONSULTING

Conoce las tablas: AdventureWorks

Sales.Customer
clientes · 19,820
CustomerID · PersonID · StoreID

Sales.SalesOrderHeader
pedidos · 31,465
SalesOrderID · CustomerID · TerritoryID

Sales.SalesTerritory
zonas de venta · 10
TerritoryID

Production.ProductCategory
categorías · 4
ProductCategoryID

Person.Person / Sales.Store
nombre de la persona o tienda
BusinessEntityID

Sales.SalesOrderDetail
líneas de pedido · 121,317
SalesOrderID · ProductID

Production.Product
productos · 504
ProductID · ProductSubcategoryID

Production.ProductSubcategory
subcategorías · 37
ProductSubcategoryID · ProductCategoryID

Llave primaria (PK): identifica cada fila y no se repite. Ej.: TerritoryID en Sales.SalesTerritory.

Llave foránea (FK): la misma columna en otra tabla, que «apunta» a la PK. Ej.: TerritoryID en los pedidos.

BLUR CONSULTING

Las tablas de la bodega para practicar

Compras · ventas en la tienda (8 filas)

id	fecha	cliente_id	producto_id	cantidad	pago
1	2026-10-01	1	1	2	Efectivo
2	2026-10-01	2	3	3	Tarjeta
3	2026-10-02	1	2	4	Efectivo
4	2026-10-02	3	4	1	Tarjeta
5	2026-10-03	NULL	5	5	Efectivo
6	2026-10-05	4	1	6	Yape
7	2026-10-05	2	5	3	Yape
8	2026-10-10	3	3	5	Efectivo

Delivery · pedidos por WhatsApp (3 filas)

id	fecha	cliente_id	producto_id	cantidad
1	2026-10-04	2	4	1
2	2026-10-06	5	2	6
3	2026-10-08	1	1	3

Clientes (5 filas)

cliente_id	nombre	distrito
1	Ana	Miraflores
2	Luis	Surco
3	Carla	Miraflores
4	Pedro	San Borja
5	Rosa	Surco

Productos (6 filas)

producto_id	producto	categoria	precio
1	Gaseosa	Bebidas	3.00
2	Agua	Bebidas	1.50
3	Arroz	Abarrotes	4.00
4	Aceite	Abarrotes	15.00
5	Galletas	Snacks	1.00
6	Chocolate	Snacks	2.50

Monto = cantidad × precio. La cantidad está en Compras y el precio en Productos: para calcularlo hay que unirlas.

Verde: llave primaria

Azul: llave foránea (apunta a otra tabla)

01. INNER JOIN: unir tablas



BLUR CONSULTING

¿Qué es un INNER JOIN?

Compras

id	cliente_id
1	1
2	2
4	3
5	NULL
6	4

Clientes

cliente_id	nombre
1	Ana
2	Luis
3	Carla
4	Pedro
5	Rosa

INNER JOIN →

Resultado (4 filas)

id	cliente_id	nombre
1	1	Ana
2	2	Luis
4	3	Carla
6	4	Pedro

Se quedan solo las parejas

La compra 5 (sin cliente) y Rosa (sin compras) no tienen pareja en la otra tabla: el INNER JOIN las deja fuera.

Definición: INNER JOIN, devuelve solo las filas que tienen coincidencias en ambas tablas. Es el tipo de join más común.

En AdventureWorks: los 31,465 pedidos se unen con Sales.SalesTerritory y cada pedido gana el nombre de su zona (Southwest, Canada, France...).

BLUR CONSULTING

Cómo se escribe un JOIN

```
SELECT h.SalesOrderID, h.TotalDue,
       t.Name AS territorio
FROM Sales.SalesOrderHeader AS h
INNER JOIN Sales.SalesTerritory AS t
        ON h.TerritoryID = t.TerritoryID
```

JOIN a secas es lo mismo que INNER JOIN.

❌ **Si no pones el alias, el motor responde:**

«Ambiguous column name 'TerritoryID'.»

1 **Alias de tabla**

h y t acortan los nombres: se escriben después de la tabla, con AS.

2 **ON dice cómo emparejar**

La llave de una tabla = la llave de la otra.

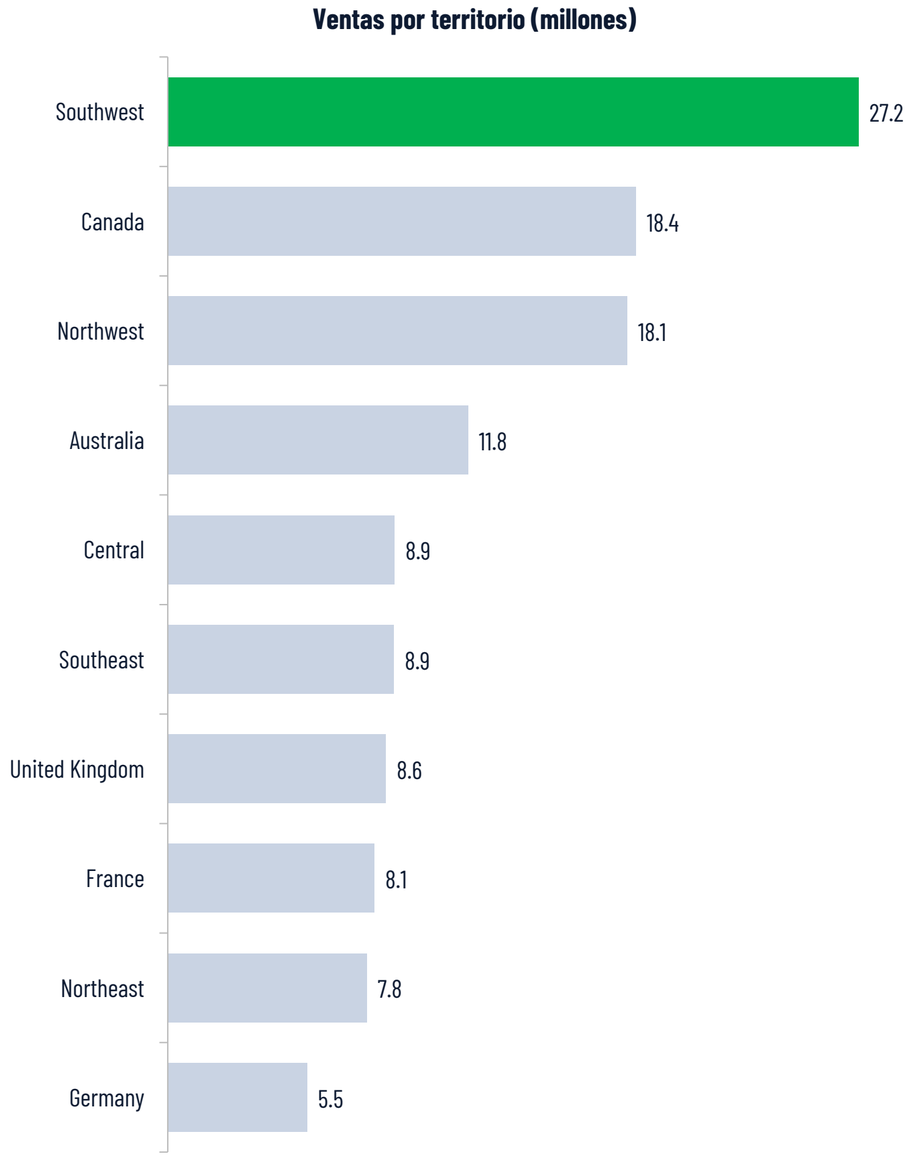
3 **Columna ambigua**

Si la columna está en las dos tablas, ponle el alias: h.TerritoryID.

Cuenta filas antes y después: hay 504 productos, pero al unirlos con su subcategoría quedan 295. Los 209 productos sin subcategoría desaparecen. (ejercicio 1.6)

BLUR CONSULTING

Insight: con nombres, el ranking se entiende



27.2 M

vende Southwest, el territorio n.º 1

57 %

de las ventas salen de los 5 territorios de EE. UU.

31,465

pedidos antes y después del JOIN: no se pierde ninguno

BLUR CONSULTING

Práctica: INNER JOIN

Compras

id	fecha	cliente_id	producto_id	cantidad	pago
1	2026-10-01	1	1	2	Efectivo
2	2026-10-01	2	3	3	Tarjeta
3	2026-10-02	1	2	4	Efectivo
4	2026-10-02	3	4	1	Tarjeta
5	2026-10-03	NULL	5	5	Efectivo
6	2026-10-05	4	1	6	Yape
7	2026-10-05	2	5	3	Yape
8	2026-10-10	3	3	5	Efectivo

Clientes

cliente_id	nombre	distrito
1	Ana	Miraflores
2	Luis	Surco
3	Carla	Miraflores
4	Pedro	San Borja
5	Rosa	Surco

Productos

producto_id	producto	categoria	precio
1	Gaseosa	Bebidas	3.00
2	Agua	Bebidas	1.50
3	Arroz	Abarrotes	4.00
4	Aceite	Abarrotes	15.00
5	Galletas	Snacks	1.00
6	Chocolate	Snacks	2.50

1.1 **Mostrar cada compra (ID) con el nombre del producto, la cantidad y el monto.**
Compras solo guarda el producto_ID; el nombre y el precio está en productos. Une las 2 tablas y calcula el monto como cantidad por precio

```
SELECT c.id, p.producto, c.cantidad, c.cantidad * p.precio AS monto
FROM Compras AS c
INNER JOIN Productos AS p ON c.producto_id = p.producto_id
```

Resultado: 8 filas · ej.: 1 Gaseosa 2 ⟶ S/ 6 · 8 Arroz 5 ⟶ S/ 20

1.2 **Mostrar cada compra (id) con el nombre del cliente. ¿Cuántas filas salen y por qué? El nombre del cliente está en Clientes y se enlaza con Compras por cliente_id. Compras tiene 8 filas: compara ese número con las filas que devuelve la consulta y revisa la columna cliente_id.**

```
SELECT c.id, cl.nombre
FROM Compras AS c
INNER JOIN Clientes AS cl ON c.cliente_id = cl.cliente_id
```

Resultado: 7 filas: la compra 5 no tiene cliente y el INNER JOIN la deja fuera

1.3 **Mostrar el monto total vendido por categoría, de mayor a menor. La categoría está en Productos. Une Compras con Productos, suma cantidad × precio de todas las compras de cada categoría y ordena del total más alto al más bajo.**

```
SELECT p.categoria, SUM(c.cantidad * p.precio) AS total
FROM Compras AS c
INNER JOIN Productos AS p ON c.producto_id = p.producto_id
GROUP BY p.categoria  ORDER BY total DESC
```

Resultado: Abarrotes S/ 47 · Bebidas S/ 30 · Snacks S/ 8

02. TIPOS DE JOIN: encontrar lo que falta



BLUR CONSULTING

Tipos de JOIN: ¿qué filas se quedan?

Clientes (izquierda)

cliente_id	nombre
1	Ana
2	Luis
3	Carla
4	Pedro
5	Rosa

LEFT JOIN →

Resultado (8 filas)

nombre	compra id
Ana	1
Ana	3
Luis	2
Luis	7
Carla	4
Carla	8
Pedro	6
Rosa	NULL

Rosa no compró en la tienda, pero sale igual: las columnas de Compras quedan vacías (NULL).

Úsalo para encontrar lo que FALTA: clientes sin compras, productos sin ventas.

INNER JOIN

Solo las filas con pareja en las dos tablas.

LEFT JOIN

Devuelve todas las filas de la tabla izquierda, y las coincidencias de la tabla derecha. Si no hay coincidencia, retorna NULLs.

RIGHT JOIN

Devuelve todas las filas de la tabla Derecha, y las coincidencias de la tabla izquierda. Si no hay coincidencia, retorna NULLs.

FULL JOIN

Todo de los dos lados. Lo que no tiene pareja sale con NULL.

BLUR CONSULTING

Cuatro reglas de LEFT, RIGHT y FULL

1 Lo que falta = IS NULL

LEFT JOIN + WHERE derecha.llave IS NULL ⟶ productos que nunca se vendieron.

```
FROM Production.Product AS p
LEFT JOIN Sales.SalesOrderDetail AS d
  ON p.ProductID = d.ProductID
WHERE d.ProductID IS NULL
```

2 Cuenta con COUNT(columna)

COUNT(*) le da 1 compra a quien tiene 0. Cuenta una columna de la tabla de la derecha.

```
SELECT cl.nombre, COUNT(c.id) AS compras
FROM Clientes AS cl
LEFT JOIN Compras AS c
  ON cl.cliente_id = c.cliente_id
```

3 El filtro de la derecha va en el ON

Si lo pones en el WHERE, el LEFT JOIN se comporta como un INNER JOIN y solo quedan 10,303 clientes de los 19,820.

```
LEFT JOIN Sales.SalesOrderHeader AS h
  ON c.CustomerID = h.CustomerID
 AND YEAR(h.OrderDate) = 2025
```

4 RIGHT = LEFT al revés

A RIGHT JOIN B da lo mismo que B LEFT JOIN A. FULL JOIN junta los dos casos.

```
FROM Production.Product AS p
LEFT JOIN Production.ProductSubcategory AS s
  ON p.ProductSubcategoryID = s.ProductSubcategoryID
```

BLUR CONSULTING

Insight:
lo que NO aparece también es información



238

productos nunca se vendieron

De 504 productos, casi la mitad (47 %) no tiene ninguna línea de venta.

701

clientes registrados sin pedidos

¿Campaña de bienvenida? Con INNER JOIN no los verías nunca.

209

productos sin subcategoría

Son piezas y componentes internos: quedan fuera de los reportes por categoría.

BLUR CONSULTING

Práctica: LEFT, RIGHT y FULL

Compras

id	fecha	cliente_id	producto_id	cantidad	pago
1	2026-10-01	1	1	2	Efectivo
2	2026-10-01	2	3	3	Tarjeta
3	2026-10-02	1	2	4	Efectivo
4	2026-10-02	3	4	1	Tarjeta
5	2026-10-03	NULL	5	5	Efectivo
6	2026-10-05	4	1	6	Yape
7	2026-10-05	2	5	3	Yape
8	2026-10-10	3	3	5	Efectivo

Clientes

cliente_id	nombre	distrito
1	Ana	Miraflores
2	Luis	Surco
3	Carla	Miraflores
4	Pedro	San Borja
5	Rosa	Surco

Productos

producto_id	producto	categoria	precio
1	Gaseosa	Bebidas	3.00
2	Agua	Bebidas	1.50
3	Arroz	Abarrotes	4.00
4	Aceite	Abarrotes	15.00
5	Galletas	Snacks	1.00
6	Chocolate	Snacks	2.50

2.1 **Muestra todos los clientes con su cantidad de compras, incluidos los que no compraron.**
Pista: FROM Clientes. No se puede perder ningún cliente.

```
SELECT cl.nombre, COUNT(c.id) AS compras
FROM Clientes AS cl
LEFT JOIN Compras AS c ON cl.cliente_id = c.cliente_id
GROUP BY cl.nombre
```

Resultado: Ana 2 · Carla 2 · Luis 2 · Pedro 1 · Rosa 0

2.2 **Muestra todos los productos con sus unidades vendidas (0 si no se vendió).**
Pista: FROM Compras. La tabla que debe salir completa va en segundo lugar.

```
SELECT p.producto, ISNULL(SUM(c.cantidad), 0) AS unidades
FROM Compras AS c
RIGHT JOIN Productos AS p ON c.producto_id = p.producto_id
GROUP BY p.producto
```

Resultado: Arroz 8 · Galletas 8 · Gaseosa 8 · Agua 4 · Aceite 1 · Chocolate 0

2.3 **Junta en una sola lista todas las compras (id) y todos los clientes (nombre). ¿Qué filas tienen NULL?**
Pista: ninguna de las dos tablas puede perder filas.

```
SELECT c.id, cl.nombre
FROM Compras AS c
FULL JOIN Clientes AS cl ON c.cliente_id = cl.cliente_id
```

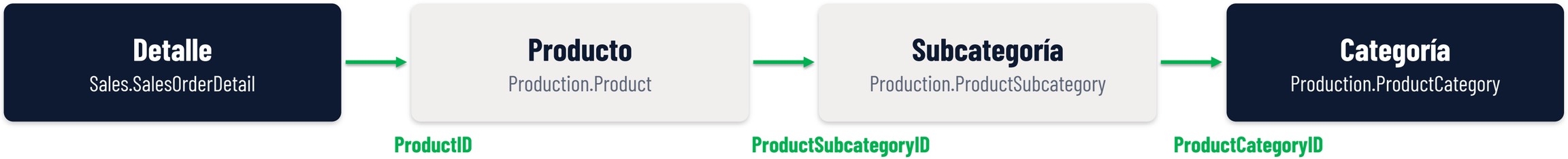
Resultado: 9 filas · la compra 5 sale sin nombre (NULL) y Rosa sale sin compra (id NULL)

03. JOIN de varias tablas



BLUR CONSULTING

Varias tablas: sigue el camino de las llaves




```sql
SELECT c.Name AS categoria, SUM(d.LineTotal) AS ingresos
FROM Sales.SalesOrderDetail AS d
INNER JOIN Production.Product            AS p ON d.ProductID = p.ProductID
INNER JOIN Production.ProductSubcategory AS s ON p.ProductSubcategoryID = s.ProductSubcategoryID
INNER JOIN Production.ProductCategory    AS c ON s.ProductCategoryID = c.ProductCategoryID
GROUP BY c.Name
ORDER BY ingresos DESC
```

categoria	ingresos
Bikes	94,651,172.70
Components	11,802,593.29
Clothing	2,120,542.52
Accessories	1,272,072.88

Truco: cada JOIN nuevo se une con una tabla que YA está en la consulta. Si te pierdes, dibuja el camino como arriba.

BLUR CONSULTING

Trampa: el JOIN repite filas

Pedidos

pedido	TotalDue
101	S/ 30

Detalle

pedido	producto	LineTotal
101	Gaseosa	S/ 6
101	Arroz	S/ 12
101	Aceite	S/ 12

Pedidos JOIN Detalle

pedido	producto	LineTotal	TotalDue
101	Gaseosa	S/ 6	S/ 30
101	Arroz	S/ 12	S/ 30
101	Aceite	S/ 12	S/ 30

SUM(TotalDue) = **S/ 90** ❌
SUM(LineTotal) = **S/ 30** ✓

Un pedido con 3 productos
sale 3 veces. Su TotalDue también.

123.2 M

ventas reales
SUM(TotalDue) en la tabla de pedidos

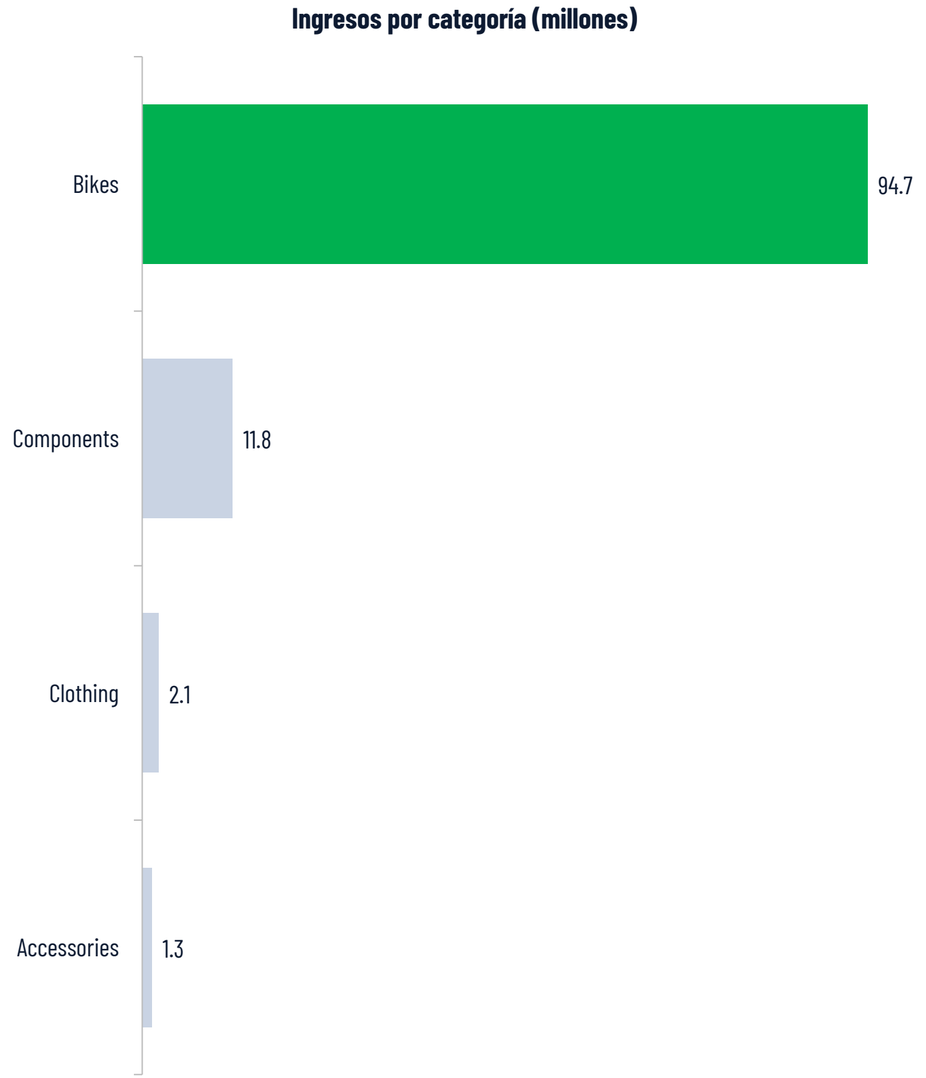
2,927.0 M

23.8 veces más
la misma suma después de unir con el detalle

Regla: suma cada número en SU tabla. TotalDue es del pedido; LineTotal es de la línea (109.8 M, sin impuestos ni envío). (Ejercicio 3.5)

BLUR CONSULTING

Insight: AdventureWorks vive de las bicicletas



86 %

de los ingresos son bicicletas

34 %

de las líneas son accesorios... pero solo el 1.2 % del dinero

5 tablas

para ver ventas por categoría y año (script 3.4)

BLUR CONSULTING

Práctica: varias tablas

Compras

id	fecha	cliente_id	producto_id	cantidad	pago
1	2026-10-01	1	1	2	Efectivo
2	2026-10-01	2	3	3	Tarjeta
3	2026-10-02	1	2	4	Efectivo
4	2026-10-02	3	4	1	Tarjeta
5	2026-10-03	NULL	5	5	Efectivo
6	2026-10-05	4	1	6	Yape
7	2026-10-05	2	5	3	Yape
8	2026-10-10	3	3	5	Efectivo

Clientes

cliente_id	nombre	distrito
1	Ana	Miraflores
2	Luis	Surco
3	Carla	Miraflores
4	Pedro	San Borja
5	Rosa	Surco

Productos

producto_id	producto	categoria	precio
1	Gaseosa	Bebidas	3.00
2	Agua	Bebidas	1.50
3	Arroz	Abarrotes	4.00
4	Aceite	Abarrotes	15.00
5	Galletas	Snacks	1.00
6	Chocolate	Snacks	2.50

3.1 **Muestra cada compra (id) con el nombre del cliente, su distrito, el nombre del producto y el monto (cantidad × precio). Pista: los datos están en 3 tablas. Une Compras con Clientes y con Productos usando sus id. ¿Por qué no aparece la compra 5?**

```sql
SELECT c.id, cl.nombre, cl.distrito, p.producto, c.cantidad * p.precio AS monto
FROM Compras AS c
INNER JOIN Clientes  AS cl ON c.cliente_id  = cl.cliente_id
INNER JOIN Productos AS p  ON c.producto_id = p.producto_id
```

Resultado: 7 filas (sin la compra 5) · ej.: 8 Carla Miraflores Arroz S/ 20

3.2 **Muestra el monto total vendido por distrito, de mayor a menor. Pista: el distrito está en Clientes y el precio en Productos. Necesitas las 3 tablas, SUM y GROUP BY.**

```sql
SELECT cl.distrito, SUM(c.cantidad * p.precio) AS total
FROM Compras AS c
INNER JOIN Clientes  AS cl ON c.cliente_id  = cl.cliente_id
INNER JOIN Productos AS p  ON c.producto_id = p.producto_id
GROUP BY cl.distrito  ORDER BY total DESC
```

Resultado: Miraflores S/ 47 · San Borja S/ 18 · Surco S/ 15

3.3 **Muestra los clientes (sin repetir) que compraron algún producto de la categoría Abarrotes. Pista: la categoría está en Productos y el nombre en Clientes. Si un cliente compró Abarrotes varias veces, ¿cómo evitas que salga repetido?**

```sql
SELECT DISTINCT cl.nombre
FROM Compras AS c
INNER JOIN Clientes  AS cl ON c.cliente_id  = cl.cliente_id
INNER JOIN Productos AS p  ON c.producto_id = p.producto_id
WHERE p.categoria = 'Abarrotes'
```

Resultado: Carla · Luis

04. UNION y UNION ALL



BLUR CONSULTING

UNION: juntar hacia abajo

SELECT ... FROM Compras

canal	fecha	cantidad
Tienda	2026-10-01	2
Tienda	2026-10-01	3
...	...	...

UNION ALL

SELECT ... FROM Delivery

canal	fecha	cantidad
Delivery	2026-10-04	1
Delivery	2026-10-06	6
Delivery	2026-10-08	3



Resultado: 8 + 3 = 11 filas

canal	fecha	cantidad
Tienda	2026-10-01	2
Tienda	2026-10-01	3
...	...	...
Tienda	2026-10-10	5
Delivery	2026-10-04	1
Delivery	2026-10-06	6
Delivery	2026-10-08	3

1 **JOIN → de costado**

Agrega COLUMNAS: a cada compra le pega el nombre del cliente.

2 **UNION → hacia abajo**

Agrega FILAS: debajo de las compras de tienda, pega las de delivery.

3 **UNION ALL vs UNION**

UNION ALL pega todo, con repetidos. UNION quita los repetidos: clientes de 2024 + 2025 = 21,448 filas con ALL y 18,666 sin repetir.

BLUR CONSULTING

Las reglas del UNION

1 Mismo número de columnas

Si arriba hay 2 columnas, abajo también.

❌ `«All queries combined using a UNION… must have an equal number of expressions»`

2 Tipos compatibles

Número con número, texto con texto, fecha con fecha.

❌ `«Conversion failed when converting the nvarchar value…»`

3 Los nombres salen de la primera

El alias se pone en el primer SELECT: 'Tienda' AS canal.

4 ORDER BY solo al final

Ordena el resultado completo, no cada parte.

INTERSECT

Lo que está en las DOS consultas: clientes que compraron en 2024 y también en 2025.

EXCEPT

Lo que está en la primera y NO en la segunda: clientes de 2024 que no volvieron en 2025.

BLUR CONSULTING

Insight: ¿cuántos clientes volvieron?

2,782

volvieron a comprar en 2025 (INTERSECT): 1 de cada 4

11,145

clientes compraron en 2024

8,363

todavía no vuelven (EXCEPT). 2025 solo llega hasta junio

Reporte por partes con UNION ALL (script 4.2)

canal	pedidos	ventas
Online	27,659	32,441,339.12
Vendedor	3,806	90,775,446.99
TOTAL	31,465	123,216,786.12

Para pensar: una lista de clientes «que no volvieron» es una lista de campaña. EXCEPT la arma en 3 líneas.

BLUR CONSULTING

Práctica: UNION y UNION ALL

Compras (tienda)

id	fecha	cliente_id	producto_id	cantidad	pago
1	2026-10-01	1	1	2	Efectivo
2	2026-10-01	2	3	3	Tarjeta
3	2026-10-02	1	2	4	Efectivo
4	2026-10-02	3	4	1	Tarjeta
5	2026-10-03	NULL	5	5	Efectivo
6	2026-10-05	4	1	6	Yape
7	2026-10-05	2	5	3	Yape
8	2026-10-10	3	3	5	Efectivo

Delivery

id	fecha	cliente_id	producto_id	cantidad
1	2026-10-04	2	4	1
2	2026-10-06	5	2	6
3	2026-10-08	1	1	3

Productos

producto_id	producto	categoria	precio
1	Gaseosa	Bebidas	3.00
2	Agua	Bebidas	1.50
3	Arroz	Abarrotes	4.00
4	Aceite	Abarrotes	15.00
5	Galletas	Snacks	1.00

4.1 **Junta en una sola lista las compras de la tienda y los pedidos de delivery: canal ('Tienda' o 'Delivery'), fecha, cliente_id, producto_id y cantidad.**
Pista: Compras y Delivery tienen columnas parecidas. Agrega una columna fija con el nombre del canal y pon un SELECT debajo del otro. ¿Hace falta quitar repetidos?

```
SELECT 'Tienda' AS canal, fecha, cliente_id, producto_id, cantidad FROM Compras
UNION ALL
SELECT 'Delivery', fecha, cliente_id, producto_id, cantidad FROM Delivery
```

Resultado: 11 filas: 8 de la tienda + 3 de delivery

4.2 **Muestra los clientes (cliente_id) que compraron en la tienda o por delivery, sin repetir y sin las compras sin cliente.**
Pista: un mismo cliente puede estar en las dos tablas. Elige entre UNION y UNION ALL según quieras o no repetidos, y filtra los NULL.

```
SELECT cliente_id FROM Compras WHERE cliente_id IS NOT NULL
UNION
SELECT cliente_id FROM Delivery
```

Resultado: 1, 2, 3, 4 y 5 (Rosa solo compra por delivery). Con UNION ALL saldrían 10 filas

4.3 **Muestra en una sola tabla el monto total vendido por canal: una fila para 'Tienda' y otra para 'Delivery'.**
Pista: calcula cada total por separado (cantidad × precio, con JOIN a Productos) y luego junta los dos resultados.

```
SELECT 'Tienda' AS canal, SUM(c.cantidad * p.precio) AS total
FROM Compras AS c INNER JOIN Productos AS p ON c.producto_id = p.producto_id
UNION ALL
SELECT 'Delivery', SUM(d.cantidad * p.precio)
FROM Delivery AS d INNER JOIN Productos AS p ON d.producto_id = p.producto_id
```

Resultado: Tienda S/ 85 · Delivery S/ 33

05. Modelamiento dimensional



BLUR CONSULTING

Data Warehouse y Data Marts



1 **Data Warehouse (DW)**

Almacén central para ANALIZAR: junta los datos de todas las áreas, guarda la historia y da una sola versión de la verdad.

2 **Data Mart**

Un pedazo del DW para UN área o proceso (ventas, finanzas). Más pequeño, más rápido de construir y de consultar.

En nuestra clase: el esquema dw con Fact_Ventas y sus 4 dimensiones es un data mart de ventas de AdventureWorks.

BLUR CONSULTING

Kimball vs Inmon

	Ralph Kimball	Bill Inmon
Enfoque	Bottom-up: de abajo hacia arriba	Top-down: de arriba hacia abajo
Empieza por	Data marts, uno por proceso de negocio (ventas, compras...)	Un Data Warehouse empresarial, para toda la empresa
Modelo	Dimensional: estrella (hechos + dimensiones)	Relacional normalizado (3NF): sin datos repetidos
Cómo integra	Bus architecture: los data marts comparten las mismas dimensiones (fecha, cliente...)	El DW central integra todo; los data marts salen de él
Primer resultado	Rápido: semanas	Lento: meses
Ideal para	Equipos que necesitan reportes pronto y crecen área por área	Empresas grandes con muchas fuentes y mucha exigencia de consistencia

BLUR CONSULTING

Enfoque híbrido: lo mejor de los dos

1. Fuentes
ERP, CRM, archivos, APIs

↓

2. Capa integrada
estilo Inmon: datos limpios, normalizados, de toda la empresa

↓

3. Data marts en estrella
estilo Kimball: un modelo por proceso, listo para analizar

↓

4. Power BI
reportes y dashboards

1 **Consistencia de Inmon**
Una sola capa central con los datos de toda la empresa, sin versiones distintas de la verdad.

2 **Velocidad de Kimball**
Los usuarios consultan estrellas simples, rápidas y fáciles de entender.

3 **Hoy: arquitectura medallion**
En la nube (Databricks, Fabric): bronce = datos crudos, plata = integrados, oro = estrellas para el negocio.

BLUR CONSULTING

Modelo estrella: hechos y dimensiones

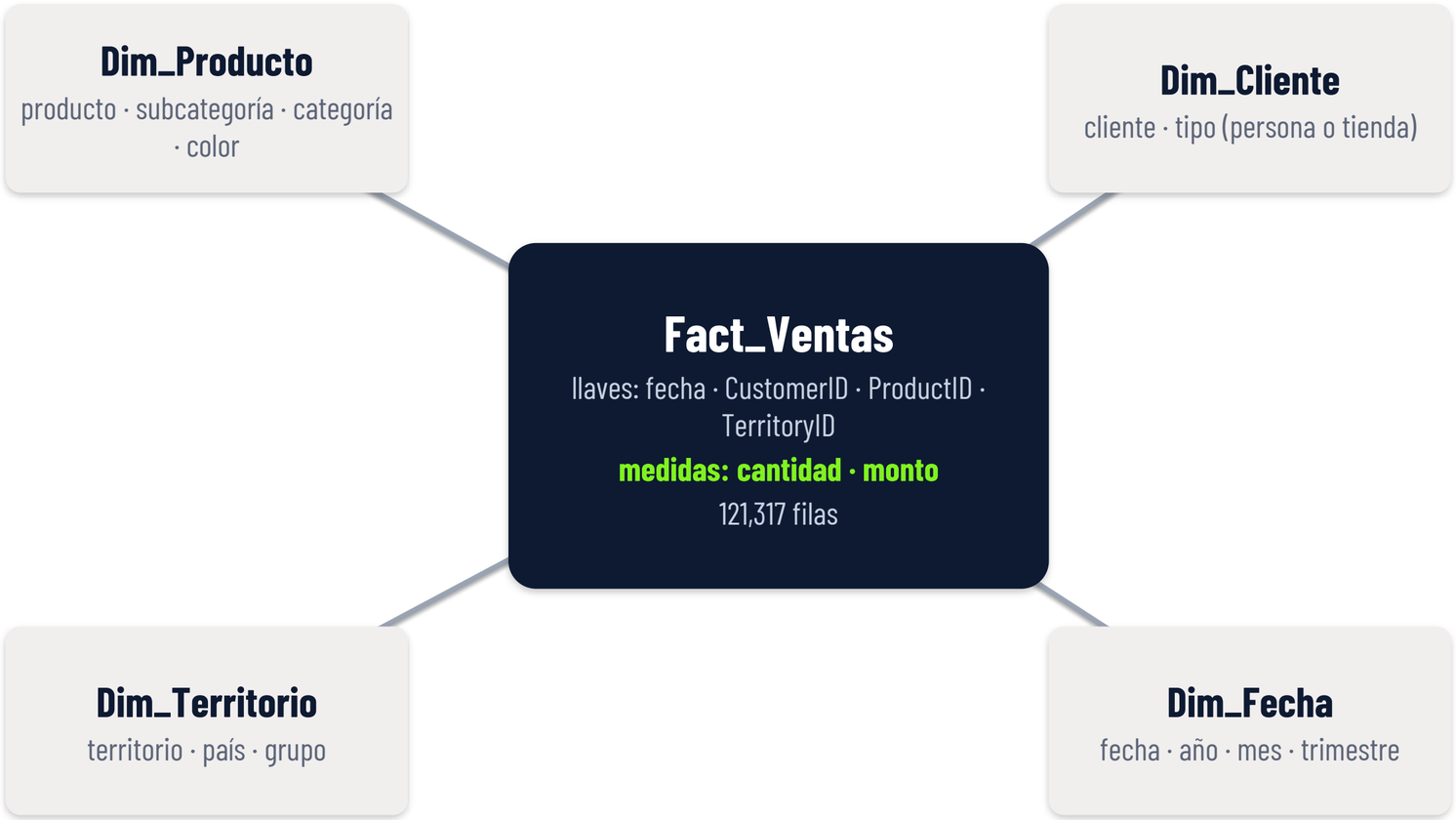


Tabla de hechos

Lo que pasó: números para sumar (cantidad, monto) y las llaves. Muchas filas.

Dimensiones

El contexto: quién, qué, dónde y cuándo. Textos para filtrar y agrupar. Pocas filas.

Grano

Qué es UNA fila de la tabla de hechos. Aquí: 1 producto dentro de un pedido.

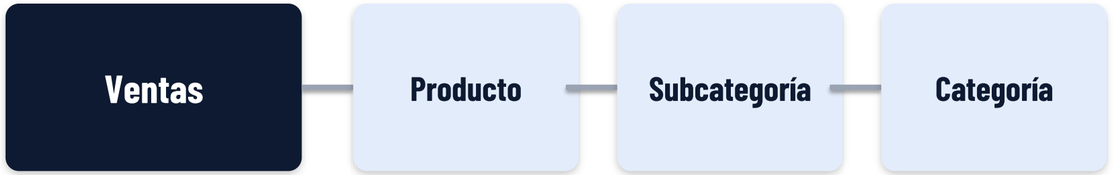
BLUR CONSULTING

Estrella vs copo de nieve

Estrella: la dimensión está aplanada



Copo de nieve: la dimensión está partida



	Estrella	Copo de nieve
Tablas por dimensión	1 (Dim_Producto)	3 (Product, Subcategory, Category)
JOIN para «ventas 2024 por país y categoría»	3	6
Textos repetidos	Sí: «Bikes» aparece 97 veces	No: «Bikes» se guarda 1 vez
Más fácil de consultar	Sí	No
Recomendado para Power BI	Sí ✓	Solo si no queda otra

Las tablas originales de AdventureWorks ya son un copo de nieve: en el paso 2 de la estrella las «aplanamos».

BLUR CONSULTING

De las tablas a la estrella, en 4 pasos

1 Define el grano

1 fila = 1 línea de pedido. Comprueba: Fact_Ventas y el detalle tienen 121,317 filas.

2 Aplana las dimensiones

Producto + Subcategoría + Categoría = Dim_Producto (con LEFT JOIN, para no perder productos).

3 Arma la tabla de hechos

Pedido + Detalle: las llaves de cada dimensión y las medidas (cantidad, monto).

4 Pregunta con un JOIN por dimensión

Ya no hay cadenas: los hechos se unen directo con cada dimensión.

```sql
SELECT t.pais, p.categoria,
       SUM(f.monto) AS ventas
FROM dw.Fact_Ventas AS f
INNER JOIN dw.Dim_Producto   AS p  ON f.ProductID = p.ProductID
INNER JOIN dw.Dim_Territorio AS t  ON f.TerritoryID = t.TerritoryID
INNER JOIN dw.Dim_Fecha      AS fe ON f.fecha = fe.fecha
WHERE fe.anio = 2024
GROUP BY t.pais, p.categoria
```

Vista = una consulta guardada con nombre. Se usa como si fuera una tabla.

En la bodega: Compras es la tabla de hechos; Clientes y Productos son sus dimensiones.

BLUR CONSULTING

Práctica: estrella y copo de nieve

Compras (hechos)

id	fecha	cliente_id	producto_id	cantidad	pago
1	2026-10-01	1	1	2	Efectivo
2	2026-10-01	2	3	3	Tarjeta
3	2026-10-02	1	2	4	Efectivo
4	2026-10-02	3	4	1	Tarjeta
5	2026-10-03	NULL	5	5	Efectivo
6	2026-10-05	4	1	6	Yape
7	2026-10-05	2	5	3	Yape
8	2026-10-10	3	3	5	Efectivo

Clientes (dimensión)

cliente_id	nombre	distrito
1	Ana	Miraflores
2	Luis	Surco
3	Carla	Miraflores
4	Pedro	San Borja
5	Rosa	Surco

Productos (dimensión)

producto_id	producto	categoria	precio
1	Gaseosa	Bebidas	3.00
2	Agua	Bebidas	1.50
3	Arroz	Abarrotes	4.00
4	Aceite	Abarrotes	15.00
5	Galletas	Snacks	1.00
6	Chocolate	Snacks	2.50

Categorias (copo de nieve)

categoria	encargado
Bebidas	Marta
Abarrotes	Jorge
Snacks	Marta

5.1 **Estrella: muestra el monto total por distrito y categoría.**
Pista: Compras es la tabla de hechos, y Clientes y Productos son sus dimensiones. Une cada dimensión directo con Compras y agrupa por las dos columnas.

```
SELECT cl.distrito, p.categoria, SUM(c.cantidad * p.precio) AS total FROM Compras AS c
INNER JOIN Clientes  AS cl ON c.cliente_id  = cl.cliente_id
INNER JOIN Productos AS p  ON c.producto_id = p.producto_id
GROUP BY cl.distrito, p.categoria
```

Resultado: Miraflores: Abarrotes S/ 35, Bebidas S/ 12 · San Borja: Bebidas S/ 18 · Surco: Abarrotes S/ 12, Snacks S/ 3

5.2 **Copo de nieve: muestra el monto total vendido por encargado.**
Pista: el encargado no está unido a Compras. Tienes que llegar en cadena: Compras → Productos → Categorias. ¿Cuántos JOIN necesitas?

```
SELECT ca.encargado, SUM(c.cantidad * p.precio) AS total FROM Compras AS c
INNER JOIN Productos  AS p  ON c.producto_id = p.producto_id
INNER JOIN Categorias AS ca ON p.categoria   = ca.categoria
GROUP BY ca.encargado
```

Resultado: Jorge S/ 47 (Abarrotes) · Marta S/ 38 (Bebidas + Snacks)

5.3 **Vista: crea la vista dbo.vw_Compras, con una fila por compra e incluyendo las que no tienen cliente. Luego, desde la vista, muestra el monto por forma de pago.**
Pista: para no perder la compra 5, ¿qué JOIN usas con Clientes? Una vez creada la vista, consúltala como si fuera una tabla.

```
CREATE OR ALTER VIEW dbo.vw_Compras AS
SELECT c.fecha, cl.nombre AS cliente, cl.distrito, p.producto, p.categoria, c.pago,
       c.cantidad * p.precio AS monto
FROM Compras AS c LEFT JOIN Clientes AS cl ON c.cliente_id = cl.cliente_id
INNER JOIN Productos AS p ON c.producto_id = p.producto_id
GO
SELECT pago, SUM(monto) AS total FROM vw_Compras GROUP BY pago
```

Resultado: Efectivo S/ 37 · Tarjeta S/ 27 · Yape S/ 21 (total S/ 85)

06. Caso integrador y práctica



BLUR CONSULTING

Caso integrador: ventas por país y categoría

EL REQUERIMIENTO

«Por país y categoría, quiero ver cuánto vendimos en 2024, cuántas unidades y cuántos clientes distintos compraron. Solo las combinaciones con más de 1 millón.»

Cómo pensarlo

1. ¿Dónde está lo que se suma? (hechos)
2. ¿Qué dimensiones necesito? (país, categoría, fecha)
3. ¿Qué filas descarto antes de agrupar?
4. ¿«Por cada» qué agrupo?
5. ¿Qué grupos descarto al final?

FROM dw.Fact_Ventas	los hechos: monto, cantidad
JOIN 3 dimensiones	Producto, Territorio y Fecha
WHERE anio = 2024	filtra filas
GROUP BY pais, categoria	un grupo por combinación
SUM · COUNT(DISTINCT)	ventas, unidades, clientes
HAVING SUM > 1 millón	filtra grupos

Resultado: 7 combinaciones pasan el millón. Las 3 primeras:

país	categoría	ventas	unidades	clientes
United States	Bikes	18,988,128.37	21,276	2,094
Canada	Bikes	4,924,929.56	5,604	393
Australia	Bikes	3,960,786.44	2,805	1,558

PRÁCTICA

Con la base de datos AdventureWorks2025

BLUR CONSULTING

1

Muestra cada producto (Name) con el nombre de su subcategoría. ¿Cuántas filas salen?

-- Pista: une Product con ProductSubcategory por ProductSubcategoryID. Compara el total con los 504 productos: ¿por qué faltan algunos?

2

Muestra cuántos clientes tiene cada territorio (por su nombre), de mayor a menor..

-- Pista: Customer tiene TerritoryID, pero el nombre está en SalesTerritory. Une las dos, cuenta clientes y agrupa por el nombre..

3

Muestra los productos que nunca se vendieron.

-- Pista: las ventas están en SalesOrderDetail. Necesitas todos los productos, se hayan vendido o no, y luego quedarte con los que no encontraron pareja (NULL).

4

Muestra los ingresos (LineTotal) por subcategoría, solo de la categoría Bikes.

-- Pista: es un copo de nieve. Llega en cadena: Detalle → Producto → Subcategoría → Categoría, y filtra por 'Bikes'.

5

Junta en una sola lista, sin repetir, los nombres de los territorios y los de los países (Person.CountryRegion).

-- Pista: algunos nombres aparecen en las dos tablas (Canada, France…). ¿UNION o UNION ALL?.

6

Muestra los clientes (CustomerID) que compraron en 2023 y no volvieron a comprar en 2024.

-- Pista: arma la lista de clientes de cada año con YEAR(OrderDate) y quédate con los del primer año que no están en el segundo.

BLUR CONSULTING

Lo que aprendimos hoy

- **INNER JOIN**
  Une por la llave y deja solo las filas que tienen pareja.
- **LEFT JOIN**
  Deja todo lo de la izquierda; con IS NULL encuentras lo que falta.
- **Varias tablas**
  Sigue el camino de llaves y suma cada número en su tabla.
- **UNION / UNION ALL**
  Apila resultados con las mismas columnas; UNION quita repetidos.
- **INTERSECT y EXCEPT**
  Lo que está en las dos listas y lo que falta en la segunda.
- **Modelamiento dimensional**
  Kimball (estrellas) e Inmon (3NF); para Power BI, estrella antes que copo.





" Lo que no se define no se puede medir.
Lo que no se mide , no se puede mejorar.
Lo que no se mejora, se degrada siempre."

William Thomson

